

22.08.2025

Diese Arbeit verbessert einen bestehenden Agenten für das Echtzeit-Strategiespiel (RTS) MicroRTS. RTS-Spiele gelten als anspruchsvolle Domäne für Deep Reinforcement Learning, da Agenten mit großen kombinatorischen Aktionsräumen, simultanen Entscheidungen sowie verzögerten und oftmals spärlichen Belohnungen umgehen müssen. Als Basismethode verwenden wir Proximal Policy Optimization (PPO). Bei vielen reinforcement learning (RL) training arten, wie bei reinem PPO kommt man an einen Punkt, an dem der Agent nicht gut nur mit PPO weiter trainiert werden kann, weil ausreichend starke Gegner, gegen die man trainieren kann fehlen oder, weil die vorhandenen Gegner sind nicht divers genug sind, um eine robuste Policy zu entwickeln. Eine naheliegende und einfache Gegenmaßnahme ist Self-Play (SP), das jedoch in der Praxis instabil sein kann und zu Überanpassung oder zyklischen Strategien führt, wodurch die Leistung gegen externe Gegner sinken kann.

League-Training (LT) stabilisiert SP, indem es eine Population aus Agenten unterschiedlicher Trainingsstände und Rollen aufbaut. Der trainierenden Agent wird gezielt gegen vielfältige Gegner antreten. Unter anderem auch gegen Main-Exploiter, die trainiert wurden, Schwachstellen der aktuellen Policy auszunutzen. Wir evaluieren die resultierenden Agenten in Matches gegen verschiedene Gegnertypen und vergleichen LT mit reinem SP sowie unserem Anfangs Agent. Der Anfangs Agent wurde mit einer Mischung aus behavior cloning (BC) und PPO Trainiert. Unsere Ergebnisse zeigen, dass LT Robustheit und Generalisierungsfähigkeit des Agenten gegenüber unterschiedlichen Gegnern erhöht.

Contents

1. Introduction	2
2. Related Work	2
3. Method	3
3.1. Environment Setup	3
3.2. Neural Network Architecture	3
3.3. Behavior Cloning	3
3.4. Reinforcement Training	3
4. Experiments	3
4.1. Setup	3
4.2. Results	3
4.3. Discussion	3
5. Conclusion	4

1. Introduction

RL etablierte sich in den letzten Jahren als leistungsstarker Ansatz für die Entwicklung von Agenten in komplexen Umgebungen. Spielumgebungen wie RTS-Spiele stellen dabei eine besondere Herausforderung dar, mit großen Aktionsräumen, langfristiger Planung und simultanen Entscheidungen. Allerdings bieten sie klar definierte Zustandsräume, Belohnungsstrukturen und wiederholbare Experimente, was sie zu idealen und herausfordernden Testumgebungen für die Erforschung von RL-Methoden macht.

Micro-RTS ist eine vereinfachte Version von RTS-Spielen, die speziell für die Forschung entwickelt wurde [Huang et al., 2021]. Sie ist diskret und reduziert die visuelle und mechanische Komplexität von großen kommerziellen RTS-Spielen. Währenddessen behält sie die Kernschwierigkeiten, wie Ressourcenmanagement, Einheitenproduktion, Kampfmechaniken und taktische Positionierung bei. Micro-RTS eignet sich deshalb für die systematische Untersuchung von Lernverfahren. Micro-RTS behält allerdings auch Trainingsherausforderungen, wie hohe kombinatorische Aktionsräume, verzögerte Belohnungen, Multi-Agent-Interaktionen und es erfordert auch robuster Strategien gegen unterschiedliche Gegner. Beispielsweise muss sich der Agent früh entscheiden, ob er gegen den aktuellen Gegner aggressiv oder defensiv spielen will, was sich auf die gesamte Spielstrategie auswirkt.

Ein Zentrales Problem in RL ist die Generalisierung gegenüber unterschiedlichen Gegnern und Strategien: Agenten lernen oft Strategien, die stark auf den Trainingsgegner oder eine spezifische Spielsituation zugeschnitten sind. Das führt zu Überanpassung und eingeschränkter Robustheit. SP, Fictitious Self-Play (FSP) und LT sind Ansätze die dieses Problem adressieren. In SP trainiert der Agent, in dem er gegen sich selbst spielt, was zu einem kontinuierlichen Lernprozess führt, der keine zusätzlichen diversen oder starken Gegner erfordert. Allerdings ist oft das Training instabil, weil es zyklisch lernt. Beispielsweise, wenn der Agentzustand B gut gegen A, C gut gegen B, und A gut gegen C ist, dann könnte der Agent in einem Zyklus gefangen sein, in dem er immer nur den Agentenzustand wieder erlernt, der gegen sich selbst gut ist, aber nicht gegen andere Gegner. Um die Instabilitäten von reinem SP zu adressieren, wird in FSP nicht gegen den aktuellen Agenten sondern gegen ältere Versionen des Agenten gespielt, was ein Zyklisches lernen verhindert und in zwei Spieler Nullsummenspielen gegen das Nash-gleichgewicht konvergiert. [Heinrich et al., 2015]. LT erweitert FSP, indem der Agent gegen historische Gegner aus einer Liga trainiert, die aus verschiedenen Agententypen besteht, beispielsweise aus historischen Trainingsständen oder

spezialisierten Strategien [Vinyals et al., 2019]. Ziel ist es, die Lernumgebung zu stabilisieren, in der Agenten gegen ein breites Spektrum an Strategien antreten und dadurch robuster werden.

Trotz der theoretischen Vorteile ist die praktische Implementierung von LT in komplexen Umgebungen wie Micro-RTS mit erheblichen Herausforderungen verbunden. Dazu gehören:

- die effiziente Verwaltung und Auswahl von Gegnern für das Training des Main-Agenten und für die spezialisierten Agenten, wie Main-Exploiter
- die Stabilisierung des Trainingsprozesses bei nicht-konstanten Gegnerverteilungen
- der Umgang mit großen Aktionsräumen und spärlichen oder verzögerten Belohnungen
- sowie die Bewertung der tatsächlichen Generalisierungsfähigkeit der trainierten Agenten.

Darüberhinaus werden wir die für die unterschiedlichen Trainingsparadigmen die Lernkurve, Strategiediversität und Robustheit vergleichen und evaluieren.

Der in diesem Projekt eingesetzte Basis-Agent, wird in einem zweistufigen Verfahren trainiert, das BC mit anschließendem Fine-Tuning durch PPO kombiniert: BC ermöglicht dem Agenten in der ersten Stufe durch Imitation von Expertenverhalten schnell eine kompetente Basisstrategie zu erlernen [Torabi et al., 2018]. Dieser Ansatz des Trainings reduziert die anfängliche, oft ineffiziente Exploration in RL [Torabi et al., 2018]. Das Lernen von Grund auf mit RL-Methoden wie PPO kann sehr zeitintensiv sein [Martin and Jhala, 2021]. PPO ist eine bekannte on-policy Policy-Optimierungsmethode, die die Policy-Update Größe über eine geclippte Surrogat-Zielfunktion begrenzt [Schulman et al., 2017]. PPO verbessert weiter die Leistung des BC-Agenten durch Exploration und Anpassung an die Umgebung, wodurch sich der Agent über die Experten Strategien hinaus anpassen und generalisieren kann [Martin and Jhala, 2021]. Dieser zweistufige Ansatz, bei dem eine mit BC vor-trainierte Policy durch einen RL-Algorithmus verfeinert wird, hat sich als effektiv erwiesen, um die Trainingszeit zu verkürzen und die Gesamtleistung im Vergleich zu einem reinen RL-Ansatz zu verbessern [Martin and Jhala, 2021].

2. Related Work

MicroRTS-Py (Gym-MicroRTS)

RAISocketAI

BeClone

AlphaStar

3. Method

3.1. Environment Setup

- T
- N
- H
- W
- C
-
-
-
-
-
-

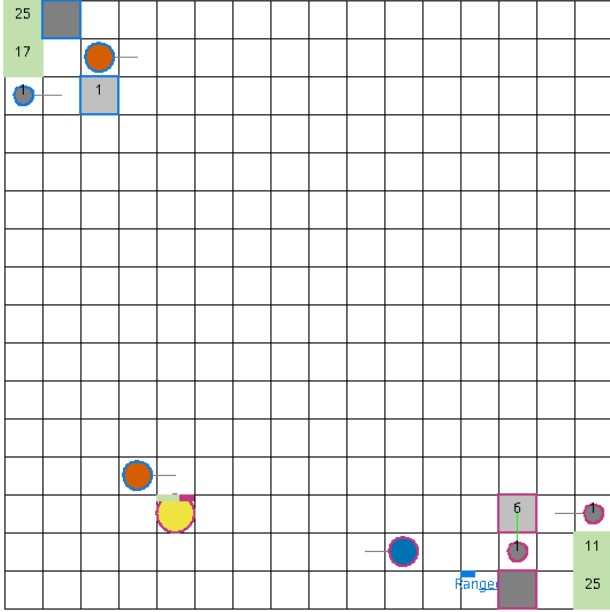


Figure 1. Screenshot basesWorkers16x16A map.

3.2. Neural Network Architecture

ScalarEncoder

$$\text{ScalarEncoder} : \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}},$$

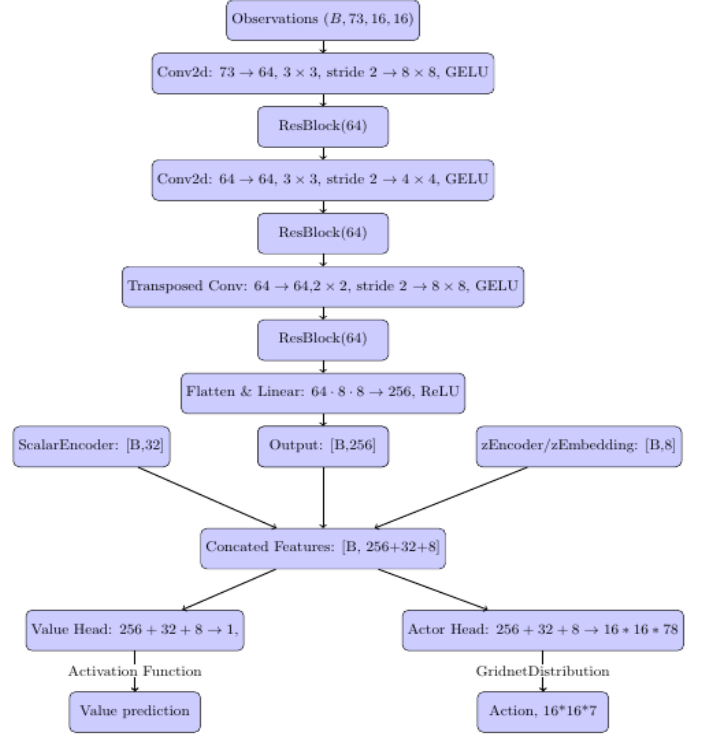


Figure 2. The Network Architecture

Table 1. Training parameters for PPO training.

Parameter	Value
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

3.3. Behavior Cloning

3.4. Reinforcement Training

4. Experiments

4.1. Setup

4.2. Results

4.3. Discussion

- **Generalization to random opponents:**
- **Map and observability constraints:**

Table 2

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

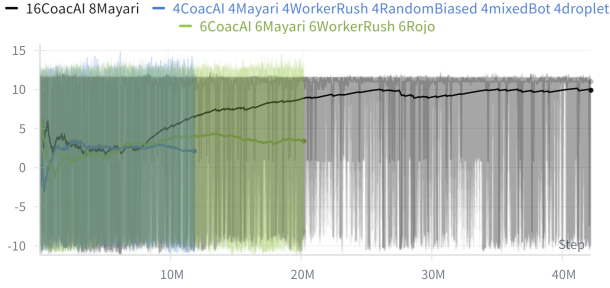


Figure 3. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the BC-FT agent.

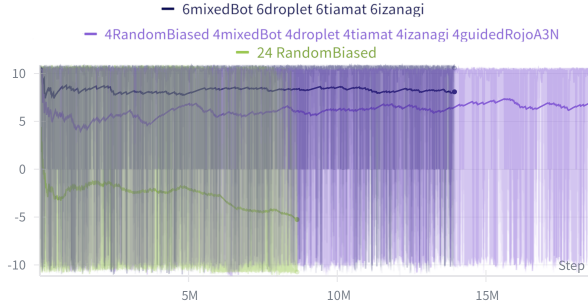


Figure 4. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the final agent.

References

- [Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 2
- [Huang et al., 2021] Huang, S., Ontanón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 2
- [Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIDE 2021 Workshops*. 2
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2
- [Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2
- [Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. 2

- Replay and training bottlenecks:
- Exploitability:

5. Conclusion

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den 22.08.2025

[illegible]